FYDP Technical Defense Handbook

Standard Academic Definitions, Subsystem Functioning & Viva Defense Rationales

Platform: Online Medical Consultation Platform | Scope: Multi-Tier Engineering & Clinical Governance | Target: FYDP Final Viva & Technical Examination

Handbook Objective: This monograph is engineered specifically for FYDP viva examinations and committee technical evaluations. Every core algorithmic model, architectural pattern, database mechanism, security control, and client-side design is presented with its formal academic definition, its exact implementation within the codebase, and direct model answers to rigorous examiner questions.

SECTION 1: AI & Machine Learning Symptom Triage Subsystem

1.1 Random Forest Classifier

MACHINE LEARNING

STANDARD ACADEMIC DEFINITION

An ensemble supervised learning method for classification and regression that operates by constructing a multitude of decision trees at training time and outputting the class mode (or probability distribution) across the individual trees.

FUNCTIONING IN THIS SYSTEM

Trained in `ai_engine/train.py` on **4,920 clinical observations** across **132 binary symptom indicators** and **41 diseases**. Configured with `n_estimators=100` trees using **Bootstrap Aggregation (Bagging)** with replacement and random feature subspacing ($\sqrt{132} \approx 11$ features per split). During triage, `predict_proba()` gathers the tree voting distribution, ranks the top-3 probable diseases, and maps the primary disease to its clinical specialty via `SPECIALTY_FOR`.

Viva Defense Answer: "*Why Random Forest instead of a single Decision Tree or Deep Neural Network?*"
A single decision tree overfits severely and has high variance. Random Forest reduces variance dramatically through bagging without increasing bias. A Deep Neural Network is inappropriate because tabular data with 4,920 records does not require millions of parameters—it would overfit, require high GPU compute, and lack the near-instant inference speed (~5ms) and interpretability of Random Forest.

1.2 TF-IDF (Term Frequency – Inverse Document Frequency)

NATURAL LANGUAGE PROCESSING

STANDARD ACADEMIC DEFINITION

A numerical weighting statistic intended to reflect how important a word is to a document in a collection or corpus, balancing local word frequency against corpus-wide rarity.

FUNCTIONING IN THIS SYSTEM

Converts unstructured free-text patient symptom descriptions (e.g., "*severe chest pain and shortness of breath*") into an L_2-normalized continuous numerical feature vector: $$\text{TF-IDF}(t, d, D) = \text{TF}(t, d) \times \log\left(\frac{N}{|\{d \in D : t \in d\}|}\right)$$ Applies sublinear scaling (`sublinear_tf=True`) to dampen repetitive words and multi-word tokenization in `ai_engine/triage.py`.

Viva Defense Answer: "*Why TF-IDF over simple Bag-of-Words (CountVectorizer)?*"
CountVectorizer counts raw word frequencies equally. In patient text, generic conversational terms like "*severe*", "*feeling*", and "*days*" appear frequently and would dominate the vector. TF-IDF down-weights ubiquitous non-discriminative terms via Inverse Document Frequency and emphasizes high-signal clinical keywords (e.g., "*jaundice*", "*rash*", "*palpitations*").

1.3 Gini Impurity (Tree Splitting Criterion)

INFORMATION THEORY

STANDARD ACADEMIC DEFINITION

A measurement of the likelihood of an incorrect classification of a new instance of a random variable, if that new instance were randomly classified according to the distribution of class labels from the dataset.

FUNCTIONING IN THIS SYSTEM

Calculated as $I_G(p) = 1 - \sum_{i=1}^{C} p_i^2$. At each split of our 100 decision trees, Scikit-learn evaluates symptom presence/absence thresholds to maximize Gini purity reduction (gain), generating clean separation between disease classes.

> **Viva Defense Answer:** "*Why Gini Impurity over Entropy (Information Gain)?*"
> Gini impurity requires no logarithmic computation ($\log_2$), making it computationally faster during tree induction while generating trees with virtually identical topological split structures and accuracy metrics.

1.4 Confidence Cutoff & Safety Fallback (< 0.35)

CLINICAL SAFETY & AI GOVERNANCE

STANDARD ACADEMIC DEFINITION

A decision-theoretic thresholding guardrail that rejects classification outputs when maximum posterior probability fails to exceed a predefined minimum certainty level.

FUNCTIONING IN THIS SYSTEM

Evaluated in `backend/triage.php`: `if (($analysis['confidence'] ?? 0) < 0.35) { $analysis['specialty'] = 'General Practitioner'; $analysis['low_confidence'] = true; }`. If confidence drops below 35% or input symptoms are ambiguous, the system flags the triage as low confidence and automatically routes the patient to General Medicine rather than risking misrouting to an inappropriate specialist.

> **Viva Defense Answer:** "*What happens if a patient enters nonsensical or rare symptoms?*"
> The platform does not guess. The low probability density automatically triggers the safety cutoff (< 0.35), categorizing the request under General Medicine for comprehensive human evaluation by a primary care physician.

1.5 Flask RESTful Microservice Architecture

DISTRIBUTED SYSTEMS

STANDARD ACADEMIC DEFINITION

A lightweight web service architectural style that decouples analytical processing into an isolated HTTP daemon communicating via stateless JSON payloads over TCP.

FUNCTIONING IN THIS SYSTEM

Defined in `ai_engine/service.py`: Runs a Flask daemon on `127.0.0.1:5000`. It receives JSON POST requests containing `{"symptoms": "..."}`, executes the model in memory, and returns disease probabilities and specialty recommendations in JSON format.

> **Viva Defense Answer:** "*Why run Python as a Flask service instead of calling Python scripts directly from PHP via shell_exec()?*"
> Invoking `shell_exec('python3 train.py')` per request forces the OS to start a new Python interpreter, import heavy libraries (NumPy, Scikit-learn), and deserialize the model file on every user click, taking 1.5–2.5 seconds. The Flask daemon holds the model in RAM, responding in under 5 milliseconds.

SECTION 2: Persistence, Relational Database & Concurrency Layer

2.1 PDO (PHP Data Objects) Singleton

SOFTWARE ARCHITECTURE & DATA ACCESS

STANDARD ACADEMIC DEFINITION

A unified database abstraction layer in PHP implementing the Singleton design pattern, ensuring that only one active database connection handle is instantiated and reused throughout a single request execution lifecycle.

FUNCTIONING IN THIS SYSTEM

Encapsulated in `backend/db.php` via `get_db_connection()`. Configures the connection with: `PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION`, `PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC`, and `PDO::ATTR_EMULATE_PREPARES => false`.

> **Viva Defense Answer:** "*Why is `ATTR_EMULATE_PREPARES => false` critical for security?*"
> When emulation is true, PHP simulates prepared statements internally by interpolating strings. When set to `false`, MySQL natively receives the SQL structure and parameter values in separate network frames. MySQL compiles the SQL grammar before values are bound as literals, mathematically preventing SQL Injection.

2.2 Database Normalization (1NF, 2NF, 3NF Proofs)

RELATIONAL DATABASE THEORY

STANDARD ACADEMIC DEFINITION

A rigorous relational schema design process that eliminates data redundancy and prevents insertion, update, and deletion anomalies by ensuring attributes depend strictly and entirely on primary keys.

FUNCTIONING IN THIS SYSTEM

1NF: All columns store atomic, non-decomposable values; zero repeating groups.
2NF: Every non-key attribute is fully functionally dependent on the primary key. Doctor-specific attributes (`specialty`, `qualification`, `pmc_number`) are segregated from base user profiles into a dedicated `doctors` entity.
3NF: Eliminates transitive dependencies. Medical department attributes (`department_name`, `description`) are isolated into `departments` rather than redundantly stored across multiple doctor records.

> **Viva Defense Answer:** "*Why choose an ACID Relational Database (MySQL InnoDB) over NoSQL (MongoDB)?*"
> Medical scheduling and prescriptions require strict ACID guarantees. In InnoDB, appointment collision locking and prescription foreign-key constraints are enforced at the storage engine level. NoSQL lacks native multi-document relational constraints, risking race-condition double bookings and dangling orphaned medical records.

2.3 B-Tree Compound Indexing Strategy

DATA STRUCTURES & QUERY OPTIMIZATION

STANDARD ACADEMIC DEFINITION

A self-balancing, sorted multi-way tree index spanning multiple table columns that provides logarithmic $O(\log n)$ search, sequential range scanning, and lookup performance.

FUNCTIONING IN THIS SYSTEM

We define two critical compound indexes:

- `idx_doctor_slot (doctor_id, appointment_date, appointment_time)` on `appointments`.
- `idx_chat_stream (appointment_id, message_id)` on `messages`.

Allows MySQL to execute slot collision verification and high-watermark chat polling via index lookups without scanning the underlying table data pages.

> **Viva Defense Answer:** "*Why does column order matter in compound indexes?*"
> B-Trees follow the Leftmost Prefix Rule. The index `(doctor_id, appointment_date, appointment_time)` allows queries filtering by `doctor_id` alone or `doctor_id + appointment_date` to utilize the index, but queries filtering solely by `appointment_time` cannot traverse the root node effectively.

2.4 Atomic Booking Collision Detection

CONCURRENCY CONTROL & RACE CONDITIONS

STANDARD ACADEMIC DEFINITION

A concurrency control pattern that ensures mutual exclusion over a shared resource (a 15-minute consultation window), preventing two concurrent threads from acquiring the same reservation.

FUNCTIONING IN THIS SYSTEM

In `backend/appointments.php`, the application executes a collision check before insertion:

```
SELECT appointment_id FROM appointments
WHERE doctor_id = ? AND appointment_date = ? AND appointment_time = ?
  AND status IN ('Pending', 'Confirmed')
```

If a record exists, the transaction fails immediately with an HTTP 409 Conflict.

> **Viva Defense Answer:** "*How do you handle two patients submitting the same slot at the exact same millisecond?*"
> The database executes each query within an isolated transactional execution unit. The first transaction acquires the slot; the second transaction encounters the committed record or unique constraint violation, aborting the duplicate booking and preserving data integrity.

SECTION 3: Backend Communication, Security & Clinical Governance

3.1 HTTP Delta-Polling (Real-Time Consultation Chat)

NETWORK COMMUNICATION

STANDARD ACADEMIC DEFINITION

An asynchronous communication protocol where a client periodically issues HTTP requests conveying a high-watermark cursor (`after_id`) to retrieve only records created after that marker.

FUNCTIONING IN THIS SYSTEM

Every 3 seconds, the browser triggers an AJAX request: `backend/messaging.php?action=get_messages&appointment_id=X&after_id=Y`. The backend executes: `SELECT * FROM messages WHERE appointment_id = ? AND message_id > ? ORDER BY message_id ASC`. If no new messages exist, the response is an empty array `[]` (< 50 bytes), maintaining low CPU and bandwidth overhead.

> **Viva Defense Answer:** "*Why HTTP Delta-Polling instead of WebSockets (Socket.io)?*"
> WebSockets require a persistent, stateful server daemon running on a non-standard port, creating firewall blocking and deployment complexity. Delta-polling is completely stateless, operates over standard HTTP/HTTPS ports (80/443), works natively with standard Apache/Nginx web servers, and provides seamless real-time chat with negligible overhead for teleconsultations.

3.2 Bcrypt Password Hashing (Cost Factor 12)

CRYPTOGRAPHY & AUTHENTICATION

STANDARD ACADEMIC DEFINITION

An adaptive, computationally intensive cryptographic key-derivation function based on the Blowfish cipher, featuring an adjustable work factor and automatic salt generation.

FUNCTIONING IN THIS SYSTEM

Employed in `backend/auth.php` via `password_hash($password, PASSWORD_BCRYPT, ['cost' => 12])`. It executes $2^{12} = 4,096$ key-expansion iterations, outputting a 60-character hash embedding the cost parameter, a 128-bit cryptographically secure random salt, and the cipher digest.

> **Viva Defense Answer:** "*Why Bcrypt over SHA-256 or MD5?*"
> SHA-256 and MD5 are general-purpose hash functions engineered for throughput, allowing modern GPUs to compute billions of hashes per second in offline dictionary attacks. Bcrypt is deliberately slow and memory-hard, making brute-force and rainbow table attacks computationally intractable.

3.3 1-to-1 Appointment-Locked Digital Prescriptions

CLINICAL GOVERNANCE & MEDICAL LAW

STANDARD ACADEMIC DEFINITION

A domain-specific relational constraint that strictly binds the generation of legal medical prescriptions to an audited, completed clinical consultation.

FUNCTIONING IN THIS SYSTEM

Enforced by a `UNIQUE(appointment_id)` foreign key constraint in the `prescriptions` table. In `backend/records.php`, the controller verifies that the active user is the assigned doctor and that the appointment status is marked `'Completed'` before permitting prescription insertion. Once recorded, prescriptions are immutable.

> **Viva Defense Answer:** "*How does this feature protect against medical malpractice?*"
> It legally binds every prescription to an audited consultation session, preventing doctors from issuing prescriptions outside of verified clinical encounters and eliminating unrecorded medication issuance.

3.4 Regulatory Medical License Verification (PMC/PMDC)

ACCESS CONTROL & REGULATORY COMPLIANCE

STANDARD ACADEMIC DEFINITION

A credential validation gate requiring medical practitioners to undergo authoritative credential verification before receiving authorization to provide healthcare services.

FUNCTIONING IN THIS SYSTEM

Newly registered doctors are saved with `is_verified = 0` and `status = 'Active'`. They are excluded from doctor directory queries (`WHERE is_verified = 1`) and triage recommendations. An Administrator reviews their Pakistan Medical Commission (PMC) license number and credentials in the Admin Portal before toggling their status to verified.

> **Viva Defense Answer:** "*Can an unverified doctor receive appointments or chat with patients?*"
> No. All public listing queries, triage recommendation queries, and appointment booking endpoints enforce `is_verified = 1`, completely blocking unapproved doctors from interacting with patients.

3.5 Append-Only Audit Logging (`system_logs`)

COMPLIANCE & NON-REPUDIATION

STANDARD ACADEMIC DEFINITION

An immutable audit logging mechanism that records transactional, security, and administrative actions with chronological timestamps and user attribution.

FUNCTIONING IN THIS SYSTEM

Implemented in `backend/db.php` via `log_audit($user_id, $action, $details)`. Automatically inserts a record into `system_logs` whenever critical events occur (user authentication, doctor verification toggle, appointment status modification, AI triage queries).

> **Viva Defense Answer:** "*Why is audit logging essential in medical software?*"
> Healthcare regulations require strict non-repudiation and auditability. If an appointment is cancelled, a diagnosis modified, or a doctor unverified, the system log provides an immutable record of who performed the action and at what exact timestamp.

SECTION 4: Client-Side Architecture, UI & Security

4.1 Vanilla JavaScript (ES6+) Architecture

FRONTEND ENGINEERING

STANDARD ACADEMIC DEFINITION

Modern ECMAScript (ES6+) executed natively by browser engines without compilation, bundling, or external framework dependencies.

FUNCTIONING IN THIS SYSTEM

Organized into modular scripts: `public/js/public.js`, `public/js/patient.js`, and `public/js/doctor.js`. Natively utilizes `async/await`, `fetch()`, DOM manipulation APIs, and modern array methods (`map`, `filter`, `reduce`) with a tiny ~25KB total footprint.

> **Viva Defense Answer:** "*Why Vanilla JS instead of React or Angular?*"
> Vanilla JS requires zero build pipeline (no Node.js, Webpack, Vite, or npm vulnerability rot). It provides near-instant First Contentful Paint (FCP) on low-spec mobile smartphones by eliminating the 300+ KB framework runtime overhead, allowing direct, surgical DOM updates without Virtual DOM diffing overhead.

4.2 Bootstrap 5 Responsive Grid & Component System

UI / UX SCAFFOLDING

STANDARD ACADEMIC DEFINITION

A mobile-first CSS framework providing a responsive 12-column flexbox grid system, standardized layout utilities, and accessible interactive interface components.

FUNCTIONING IN THIS SYSTEM

Enforces cross-platform layout responsiveness between doctor clinical desktop monitors and patient mobile phones using grid classes (`container`, `row`, `col-md-6`, `col-xl-4`), accessible modals (appointment booking, prescription drafting), and clinical status badges.

> **Viva Defense Answer:** "*Why Bootstrap over custom CSS or Tailwind?*"
> Writing cross-browser responsive grids and accessible modal backdrops from scratch is error-prone. Tailwind requires a PostCSS build compilation step. Bootstrap 5 delivers a battle-tested, accessible UI layer via a single stylesheet with zero build requirements.

4.3 Dynamic 15-Minute Slot Generation & Disabling

ALGORITHMS & SCHEDULING

STANDARD ACADEMIC DEFINITION

An algorithmic time-discretization routine that divides continuous clinical operating hours into discrete, non-overlapping consultation windows.

FUNCTIONING IN THIS SYSTEM

Implemented in JavaScript: Loops from start time (e.g. 09:00) to end time (e.g. 17:00) in 15-minute step intervals. Cross-references the doctor's already-booked appointments for the chosen date retrieved via AJAX. If a slot matches an existing booking, the button is rendered with `disabled` and the CSS class `slot-booked`.

> **Viva Defense Answer:** "*What happens if two patients open the calendar at the exact same second?*"
> Even if both users see an open slot in the frontend UI, the backend atomic check (`SELECT ... WHERE status IN ('Pending', 'Confirmed')`) rejects the second submission at the database transaction level with an HTTP 409 Conflict, preventing race conditions.

4.4 DOM Sanitization & XSS Prevention (`escapeHtml`)

APPLICATION SECURITY

STANDARD ACADEMIC DEFINITION

A defensive client-side encoding routine that translates untrusted characters into safe HTML character entities prior to DOM insertion, mitigating Cross-Site Scripting (XSS).

FUNCTIONING IN THIS SYSTEM

Defined in `public/js/public.js`:

```
function escapeHtml(str) {
  return String(str || '').replace(/&/g, '&amp;')
    .replace(/</g, '&lt;').replace(/>/g, '&gt;')
    .replace(/"/g, '&quot;').replace(/'/g, '&#039;');
}
```

Every doctor note, chat message, and diagnosis string is sanitized through `escapeHtml()` before rendering into `innerHTML`.

> **Viva Defense Answer:** "*How do you prevent stored XSS attacks via consultation chat messages?*"
> Even if an attacker posts `<script>alert(document.cookie)</script>`, `escapeHtml()` converts it to `<script>...`, causing the browser to render it purely as visible, harmless text without executing JavaScript.

4.5 Doctor Seniority Ranking (`experience_years DESC`)

TRIAGE & DECISION SUPPORT

STANDARD ACADEMIC DEFINITION

A deterministic clinical sorting algorithm prioritizing medical practitioners based on verified years of active practice.

FUNCTIONING IN THIS SYSTEM

When triage identifies a matching specialty, doctors are retrieved using: `SELECT ... WHERE specialty = ? AND is_verified = 1 ORDER BY experience_years DESC LIMIT 3`. If no specialists exist, it falls back to the top-2 most experienced General Practitioners.

> **Viva Defense Answer:** "*Why rank by experience instead of patient reviews/ratings?*"
> In clinical triage, safety is paramount: acute conditions must be routed to senior clinicians. Rating systems suffer from the **cold-start problem** (newly registered expert doctors have 0 reviews). Experience is an objective, verifiable credential.

SECTION 5: Viva Rapid-Recall Master Comparison Matrix

TECHNOLOGY / CONCEPT	STANDARD ACADEMIC DEFINITION	SYSTEM FUNCTIONING	CORE VIVA DEFENSE POINT
Random Forest	Ensemble of decision trees with bagging and feature subspacing	Classifies 132 symptoms into 41 diseases using 100 trees with probability ranking	Prevents single-tree overfitting; fast ~5ms inference on tabular clinical data
TF-IDF	Term Frequency balanced against Inverse Document Frequency	Vectorizes free-form patient text with sublinear scaling and L2 normalization	Penalizes common conversational stop-words; boosts discriminative clinical keywords
Gini Impurity	Probability of misclassifying a randomly selected instance	Splitting metric in decision trees evaluating symptom presence thresholds	Computationally faster than Entropy (no log calculation) with identical split topology
Confidence Cutoff	Decision-theoretic thresholding gate rejecting low posterior probabilities	Evaluates if confidence < 0.35; defaults ambiguous triage to General Medicine	Prevents AI hallucination / false certainty; acts strictly as an assistive triage aid
Flask Microservice	Decoupled HTTP service handling analytical ML requests via JSON	Hosts Python inference daemon on `127.0.0.1:5000`; accessed via PHP cURL	Holds model in RAM for ~5ms response; avoids costly script re-initialization
PDO Singleton	Creational pattern providing a single unified database connection instance	Manages MySQL connection in `backend/db.php` with disabled emulation	Separates SQL compilation from parameter binding, eliminating SQL Injection
B-Tree Indexes	Self-balancing multi-way sorted tree maintaining $O(\log n)$ lookup time	Compound indexes on `(doctor_id, date, time)` and `(appointment_id, message_id)`	Replaces full-table scans with fast logarithmic index lookups during booking and chat
Delta-Polling	Asynchronous client polling requesting records newer than `after_id`	Queries chat messages every 3 seconds with high-watermark pagination	Stateless, firewall-proof (ports 80/443), and avoids complex WebSocket daemons
Bcrypt (Cost 12)	Adaptive cryptographic key-derivation based on Blowfish cipher	Hashes user passwords with $2^{12} = 4,096$ key-expansion iterations and salt	Slow by design, neutralizing GPU-accelerated dictionary and rainbow table attacks
1-to-1 Prescription	Domain constraint binding prescriptions to completed consultations	`UNIQUE(appointment_id)` foreign key constraint; immutable once written	Enforces legal medical accountability; prevents issuing unrecorded prescriptions
PMC Verification	Regulatory administrative gate authenticating doctor medical licenses	Stores `is_verified = 0` on registration; toggled after Admin document review	Blocks unverified practitioners from patient directories and consultation booking
Vanilla JS (ES6+)	Native ECMAScript running directly in browser without build pipelines	Powers async API orchestration, chat polling, and booking slot UI (~25KB bundle)	Zero npm fragility; eliminates framework bundle overhead for instant mobile loading
Doctor Seniority	Sorting clinical practitioners based on verified years in clinical practice	`ORDER BY experience_years DESC LIMIT 3` for matched specialty doctors	Deterministic clinical safety routing; avoids cold-start bias of patient reviews